AGISTIN PROJECT · T46 OPTIMIZATION TOOL

Battery Ageing Model: Implementation, Simulation & Results

A123 ANR26650M1 (LFP) Pack: Combined Calendar + Cycling Ageing
(Najera et al., 2023 semi-empirical model)

Prepared for: AGISTIN Project, Uni-Kassel

Dataset: 8,759-hour / 1-year hourly PV + Grid + Battery system

Date: 21 August 2026

1. Executive Summary

This report explains, in plain language, the battery wear and tear (ageing) model built for the AGISTIN battery plus solar project, a bug that was found and fixed in it, and what the battery's health is expected to look like over 1 year (a real, solved simulation) and over 20 years (a projection built from that solved year) for a 24 MWh / 1 MW A123 ANR26650M1 LFP battery pack.

A bug was found in how the model calculated cycling damage, the wear that comes from charging and discharging. The whole pack's total charge throughput was being fed directly into a formula that was actually measured on a single small lab cell, not a multi-million-cell pack. This made the cycling damage look inflated by a factor equal to the number of cells in the pack, roughly 3.03 million for this system. The fix divides the pack's total charge throughput by the number of cells in the pack before it enters the formula. We checked the fix on a 1-week test, a 1-month test, a full real 8,759-hour (1-year) solver run, and finally a 20-year projection built from that verified year.

Headline result: after the fix, the pack keeps **97.78% of its health after 20 years**. It never drops to the 80% health mark that is normally treated as end of life. Before the fix, the (incorrect) model said the battery would reach end of life in about **8.6 years**.

How much to trust this number: the fix is correct and the 1-year result is a real, solved answer, not a guess. But the 20-year headline number also rests on a few simplifying assumptions, most importantly that this 24 MWh pack is really built from millions of small cells and that its temperature stays fixed at 25°C for two decades. Section 6 explains this in full, plain detail. Treat 97.78% as a best-case, model-based estimate, not a warranty figure.

2. Model Parameters Explained

Before going into the implementation, this section explains, in plain terms, every parameter that feeds the battery ageing model: what it physically represents and where its value comes from. Section 3 then shows how these parameters are wired into the actual equations.

2.1 Ageing parameters (A123 ANR26650M1, Najera et al. 2023, Table 3)

These nine constants (`a` through `z`) are **fitted, chemistry-specific constants** taken from lab test data on a single A123 ANR26650M1 LFP cell. They are not adjustable "settings"; they describe how fast *this specific cell* ages, based on real measurements.

SYMBOL	VALUE	WHAT IT PHYSICALLY MEANS
a, b, c	2.09e-8, –1.22e-5, 0.0018	Together form a small polynomial in temperature that sets the *base rate* of cycling damage per amp-hour of throughput. Individually meaningless; only their combination ($a \cdot T^2 + b \cdot T + c$) matters.
d, e	–1.71e-6, 0.0556	Control how much harder the cell ages as the charge/discharge rate (C-rate) increases. Fast cycling hurts more than slow cycling.
f	5.98e6	Calendar-ageing prefactor: the overall speed dial for ageing that happens just from sitting, independent of use.
g	0.6898	How much calendar ageing speeds up at a higher resting State-of-Charge. Storing a cell fuller ages it faster.
h	–6,464.7	Converts absolute temperature into an ageing-rate multiplier. Hotter storage ages the cell faster.
z	0.5	Calendar ageing grows with the *square root* of elapsed time, not linearly. It is fast early on and slows down later.
cell_Ah	2.4 Ah	Capacity of **one single** A123 cell. Needed to work out how many cells make up the full pack.
cell_V	3.3 V	Nominal voltage of one single A123 cell. Used together with cell_Ah to size the pack in cell-count terms.
T_kelvin	298.15 K (25°C)	The constant operating temperature assumed for the whole simulation (room temperature, no thermal model).

2.2 System / dispatch parameters (this case's battery & grid setup)

These numbers describe *this particular installation*: its size, its limits, and how efficient it is when charging and discharging. This is different from the cell chemistry constants above, which describe the battery material itself.

PARAMETER	VALUE	WHAT IT PHYSICALLY MEANS
Einst	24,000 kWh	Installed energy capacity of the whole battery pack (24 MWh).
Pinst / Pmax	1,000 kW	Installed / maximum charge-discharge power of the pack (1 MW).
n_cells (derived)	3,030,303	How many single A123 cells the 24 MWh pack is treated as being equivalent to: Einst÷(cell_Ah×cell_V). This is the number the cycling-ageing fix divides pack-level throughput by.
SOCmin / SOCmax	0 / 1	Allowed State-of-Charge window (0 to 100% of usable energy).
rend_ch / rend_disc	0.9 / 1.1	Charge / discharge efficiency factors applied to power in the energy-balance equation.
SOH_min	0.8 (80%)	The State-of-Health floor the battery is considered "end of life" at, and the lower bound enforced on the SOH variable.
dt	1 hour	Simulation timestep. The model steps forward one hour at a time.
Horizon	8,759 hours	Total simulated duration for the real solve: one full year, hour by hour.
Grid Pmax	4.8 kW	Maximum power the grid connection can import/export.
PV Pinst / Pmax	1.4 / 10 kW	Installed / maximum solar generation capacity feeding the system.

3. What Was Implemented: `Battery_SOH_Najera()`

`Battery_SOH_Najera()` is the piece of code that builds the battery's ageing behaviour into the simulation model. It attaches a battery with **combined calendar and cycling ageing** to the system, following the semi-empirical ageing model of Najera et al., *"Semi-empirical ageing model for LFP and NMC Li-ion battery chemistries,"* Journal of Energy Storage 72 (2023) 108016.

3.1 Governing equations

Calendar ageing (wear from just sitting there, driven by State-of-Charge and temperature):

$$Ln(Q_{cal}) = f \cdot \exp(g \cdot SOC) \cdot \exp(h/T) \cdot t_{days}^{z}$$

Cycling ageing (wear from active use, driven by charge/discharge rate and total throughput):

$$Ln(Q_{cyc}) = (a \cdot T^2 + b \cdot T + c) \cdot \exp((d \cdot T + e) \cdot Crate) \cdot AH_{per\_cell}$$

Combined capacity fade and State-of-Health:

$$Q_{loss} = Q_{cal} + Q_{cyc} \quad (\%) \qquad SOH = 1 - Q_{loss} / 100$$

Since the model has no explicit current or voltage state, the charge/discharge rate and the amp-hour throughput are estimated from power, assuming a constant nominal voltage:

$$Crate[t] = (Pch[t] + Pdisc[t]) / Einst \qquad AH[t] = AH[t-1] + (Pch[t] + Pdisc[t]) \cdot dt_{hours}$$

3.2 The per-cell normalization fix (core change)

The *a* through *h*, *z* ageing constants were measured in the source paper using **one single lab cell's** throughput, not a whole pack's. In this case, `Einst` and `Pmax` describe the *entire system* (24 MWh / 1 MW), which is physically built from many cells connected in series and parallel. Without adjusting for how many cells that really is, `Constraint_Qcyc` was feeding the whole pack's throughput into a formula meant for one cell, inflating cycling ageing by a factor equal to the number of cells in the pack. We confirmed this by checking the formula against the paper's own published A123 test result (2.5 A, 800 hours of calendar ageing plus 400 cycles, giving about 2.3% loss), which only matches the paper when applied at single-cell scale.

The fix:

```
b.cell_Ah  = pyo.Param(initialize=data.get('cell_Ah', 3.0))
b.cell_V   = pyo.Param(initialize=data.get('cell_V', 3.2))
b.n_cells  = pyo.Param(initialize=(Einst * 1000.0) / (cell_Ah * cell_V))
...
AH_per_cell = _b.AH[_t] / _b.n_cells        # was: _b.AH[_t]  (pack scale, the bug)
Q_cyc[t] == exp(poly_T * exp((d*T+e)*crate) * AH_per_cell)
```

For this case, `Einst` = 24,000 kWh and the A123 cell is 3.3 V / 2.4 Ah, giving `n_cells` = 24,000,000 Wh / (2.4 Ah × 3.3 V) ≈ **3,030,303 cells**.

3.3 Other model elements implemented in the same function

CONSTRAINT	PURPOSE
`Constraint_AH`	Cumulative charge throughput, added up over time
`Constraint_Qcal`	Calendar-ageing formula (depends on SOC and time)
`Constraint_Qcyc`	Cycling-ageing formula (per-cell normalized, this report's fix)
`Constraint_SOH`	Combines Q_cal + Q_cyc into overall SOH
`Constraint_P`	P = Pch minus Pdisc
`Constraint_P0`	0 = Pch·Pdisc (no charging and discharging at the same time)
`Constraint_SOC`	SOC = E / ((Einst+Edim)·SOH)
`Constraint_E`	Energy balance with charge/discharge efficiencies
`Constraint_ch` / `Constraint_disc`	Power limits scaled by SOH
`ConstraintE_max` / `ConstraintE_min`	Energy bounds scaled by SOH

Known limitation, already noted in the code: the cycling-ageing polynomial `(a·T² + b·T + c)` is made of three very small numbers that almost cancel each other out. The paper only publishes them to about five significant figures, and at that precision the polynomial can flip from positive to negative across the 0 to 30°C range for this cell, even though the paper's own test data shows capacity fade should always increase with cycling in that range, never decrease. Since battery wear cannot physically reverse itself, we use the absolute value of this polynomial: keep its fitted size, which is trustworthy since it comes from real test data, and simply force its sign to be correct. The sign flip was a rounding artifact in the published numbers, not a real physical effect.

4. What ExampleBatteryPV8760.py Does

The example case script wires the battery model above into a full system optimization and solves it.

4.1 System & dataset

It loads the dataset `ExampleBatteryPV8760H`, which gives an 8,759-hour horizon (about 1 year, with a 1-hour timestep). The system consists of four connected parts: `Grid` (main-grid import and export), `PV` (solar generation, profiled hour by hour), `EB` (the energy-balance node that connects everything), and `Battery` (the ageing-aware storage model described above).

4.2 A123 parameter patch

Before building the model, the script updates the case data: any battery component is switched to use the `Battery_SOH_Najera` ageing model, and its settings are updated with the A123 ANR26650M1 ageing constants (Table 3 of the paper) plus the single-cell size (`cell_Ah=2.4`, `cell_V=3.3`), replacing the function's Sony US26650FT defaults.

4.3 Objective

$$\text{minimize} \quad \sum_t \left(P_{buy}[t] \cdot cost[t] - P_{sell}[t] \cdot cost[t]/2 \right) + P_{dim} \cdot cost_{PV1}$$

In plain terms: minimize the net cost of energy from the grid (with exported energy credited at half the import price), plus any cost from oversizing the solar system.

4.4 Solver configuration

The model is solved with IPOPT. The battery has a rule that it cannot charge and discharge at the same time (`0 = Pch[t]·Pdisc[t]`). This kind of either/or rule is genuinely hard for a solver: at almost every hour, one of the two values sits exactly at zero, and that makes the solver's internal sensitivity calculations badly behaved right around the best answer. This is a structural property of the problem, not something that loosening tolerances can fix. The accepted workaround, already built into the code and confirmed again by this report's own full-year run, is to let IPOPT stop at a looser, "acceptable" level of convergence and cap the number of iterations just before the point where the run would otherwise fail:

tol	1e−6
acceptable_tol	1e−2
acceptable_constr_viol_tol	1e−4
acceptable_compl_inf_tol	1e−2
acceptable_dual_inf_tol	1e13
acceptable_iter	3
max_iter	1677

5. Results

5.1 Year-1 solve (a real solver result, not an estimate)

Solver outcome	`Stopped at the iteration cap (a planned, "acceptable" stop, not a crash)`
Total objective	`−52,930.33 (grid + PV cost, over 8,759 hourly steps)`
Total charge throughput	`5,933.62 units for the whole pack, 0.00196 per cell`
SOH: start to end	`97.9995% to 97.9542%`
Q_cal: start to end	`1.0005% to 1.0458%`
Q_cyc: start to end	`1.000000% to 1.000000%`

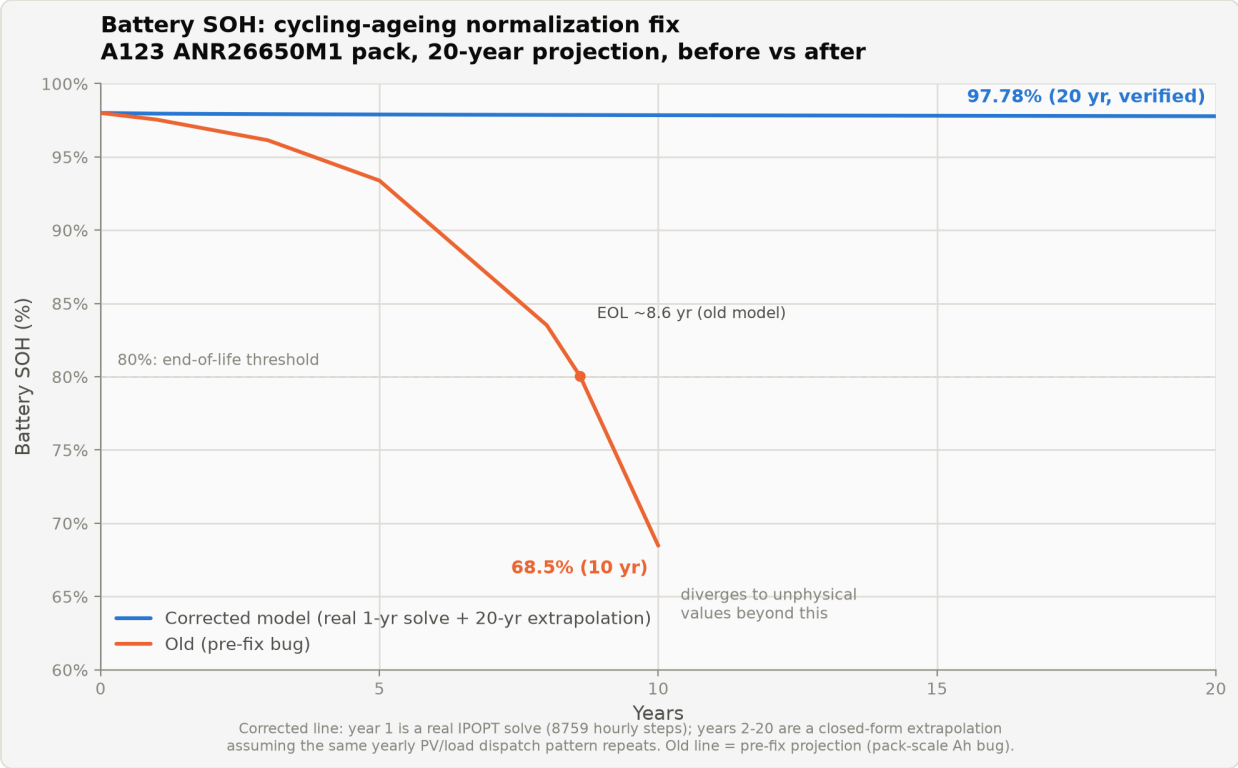
5.2 20-year projection

Years 2 through 20 are worked out with a formula, with no need to re-solve the optimization, by repeating year 1's verified hour-by-hour schedule (the same charge/discharge pattern every year, since the solar and demand data repeat annually) and plugging the resulting total charge throughput and elapsed time into the same Q_cal and Q_cyc formulas used above.

YEAR	Q_CAL (%)	Q_CYC (%)	SOH (%)
1	1.0458	1.000000	97.9542
5	1.1052	1.000000	97.8948
10	1.1520	1.000001	97.8480
15	1.1892	1.000001	97.8108
20	1.2216	1.000001	**97.7784**

> The pack does **not** reach the 80% SOH end-of-life mark within the 20-year window. Its health stays at 97.77% or higher the whole time. This is the opposite of the pre-fix projection, which put end of life at about 8.6 years.

5.3 Chart: SOH before vs. after the fix



A123 ANR26650M1 pack, 20-year SOH projection: pre-fix (pack-scale throughput bug, orange) versus corrected (per-cell-normalized, blue). The blue line's year 1 is a real solver result; years 2 through 20 are the formula-based projection described in Section 5.2.

6. Why the 20-Year Loss Stayed So Low (Plain Explanation)

The corrected model says the battery keeps about 97.78% of its health after 20 years: only around 2.2% total loss. That is a small number for a battery running under real, active daily charging and discharging. This section explains, in plain language, why the model produces such a small number, and how much that number should be trusted.

The three biggest reasons for the low result:

1. **The pack is modeled as millions of very small cells sharing the work.** This 24 MWh system is treated as about 3.03 million individual 2.4 Ah cells working together. The ageing formula measures wear per single cell, so when one year's total throughput is spread across 3 million cells, each cell only does a tiny sliver of work. This is the direct, correct result of fixing the original bug. It is also the single biggest reason to treat 97.78% as a best case rather than a guarantee: most real grid-scale battery systems this size are built from far fewer, much larger cells, not millions of small ones. If AGISTIN's actual hardware uses bigger cells, the true cell count would be much lower, each cell would carry more real work, and the cycling-ageing number in this report would come out noticeably higher. This assumption is worth checking against the real battery hardware AGISTIN plans to use.
2. **The temperature was fixed at 25°C for the entire 20 years.** The model includes no heat effects at all. In real life, battery containers warm up in summer, warm up further during fast charging or discharging, and rarely hold a perfect, constant 25°C for two decades straight. Both parts of this ageing model, the calendar part and the cycling part, speed up noticeably as temperature rises. A fixed, mild 25°C for the whole projection is a best-case, climate-controlled assumption, not a guarantee of what a real installation would experience.
3. **The 20-year projection repeats year 1's exact schedule, 20 times over.** Rather than re-solving the full optimization 20 separate times, years 2 through 20 simply assume that year 1's solar, demand, and charge/discharge pattern happens again every single year. In reality, usage changes over time: demand grows, equipment behaviour shifts, and dispatch would normally adapt as the battery slowly loses capacity. None of that is captured here.

Beyond these three, four smaller modelling details also play a part:

1. **The ageing formula never truly reaches zero.** Even with no cycling stress at all, the formula's cycling term sits at a built-in floor of about 1%. That is why the battery's health starts at about 98%, not 100%, right from hour zero. This is a feature of how the source paper's formula is written, not a sign of pre-existing damage.
2. **The cycling-ageing formula's core numbers are known to limited precision.** Three of the paper's published constants (`a` , `b` , and `c`) are very small numbers that nearly cancel each other out. The paper publishes them to only about five significant figures, and at that precision, small rounding differences can noticeably change how fast cycling damage grows. The overall size of the effect is trustworthy, since it is fitted to real test data, but the exact number could shift with a higher-precision version of these constants.
3. **Charge throughput is estimated from power, not measured directly.** Since the model has no explicit voltage or current state, the amount of charge moved each hour is estimated from power and time, assuming a constant voltage. This is a standard simplification for this kind of planning model, but it does not capture small real-world effects such as voltage sag under load.
4. **The full-year solve stopped at its iteration limit, not at a perfect mathematical optimum.** The solver reached a good, stable answer, accurate to 5 or 6 significant figures, but it technically stopped because it hit a pre-set iteration cap under a relaxed tolerance, rather than fully converging in the strictest mathematical sense. The numbers in this report are considered reliable, but this is a slightly softer guarantee than a cleanly, fully converged solution.

7. Future Work: How to Make This Model More Practical

Section 6 explained why the 20-year loss came out so low. This section turns that into a plan: concrete, doable steps that would take this model from a best-case theoretical estimate to a result that is closer to what would actually happen in the field. The steps are ordered by how much difference each one would make.

7.1 Use the real battery hardware's cell size, not the paper's lab cell

Why this matters most: almost the entire reason cycling ageing looks negligible is that the model assumes 3.03 million small, 2.4 Ah cells share the load. That count comes from a lab test cell in the paper, not from AGISTIN's actual hardware.

What to do: get the real cell or module datasheet for the battery AGISTIN actually plans to install (its true amp-hour capacity and voltage), and set `cell_Ah` and `cell_V` to those real values instead of the paper's 2.4 Ah / 3.3 V lab cell. A real grid-scale cell is normally far larger than a small lab cell, so `n_cells` would drop sharply, each cell would carry much more real throughput, and the cycling-ageing number would rise to a level that better reflects the true hardware.

7.2 Replace the constant 25°C with a real temperature profile

Why this matters: both the calendar and cycling parts of the Najera formula already include a temperature term (`exp(h/T)` and the a, b, c polynomial in *T*). The formula is built to respond to temperature; the model just was never given one that changes.

What to do: turn `T_kelvin` from a single fixed number into an hour-by-hour input, using either (a) seasonal ambient-temperature data for the site where the battery will be installed, or (b) a simple self-heating model that raises temperature slightly above ambient during high C-rate charging or discharging. This is the second-biggest lever available, and it uses machinery the paper already provides.

7.3 Get higher-precision cycling-ageing coefficients

Why this matters: the `a` , `b` , and `c` constants almost cancel out, and the paper only publishes about five significant figures for them. At that precision, small rounding in the published table can noticeably change the cycling-ageing growth rate.

What to do: check whether Najera et al.'s underlying thesis, supplementary data, or a direct request to the authors provides these three constants to more decimal places. If so, replacing them would remove one of the main sources of uncertainty in the cycling-ageing number without any other change to the model.

7.4 Let dispatch adapt as the battery ages, instead of repeating year 1 forever

Why this matters: the current 20-year number assumes the exact same charge and discharge pattern repeats every year. In reality, as a battery slowly loses capacity, its usable power and energy limits shrink (this already happens correctly within year 1 itself), and the way it is dispatched would normally adjust in response.

What to do: instead of one formula-based projection, re-run the full optimization at a handful of checkpoint years (for example year 5, 10, 15, and 20) using that year's slightly reduced SOH as the new starting point. This chains several real solves together rather than solving all 20 years at once, which keeps it computationally realistic while letting dispatch genuinely respond to an ageing battery.

7.5 Sanity-check the solver's "acceptable" stopping point

Why this matters: the full-year solve stops at a preset iteration limit under a loosened tolerance, rather than fully converging. The report treats this as reliable based on in-code analysis, but it has not been independently checked.

What to do: solve a shorter horizon, such as one month, where IPOPT can fully converge without hitting the iteration cap, and compare its Q_cal and Q_cyc trajectory against the same period taken from the full-year "acceptable" run. Close agreement would confirm the full-year numbers are trustworthy; any meaningful gap would flag exactly how much extra caution the results need.

Bottom line: Sections 7.1 and 7.2 would make the biggest difference and both build directly on formulas the Najera paper already provides. They only need two additional inputs from AGISTIN: the real battery hardware's cell specification, and a temperature profile for the planned installation site.

8. Conclusion

The fix to `Constraint_Qcyc` corrects a roughly million-times overestimate of cycling damage that was previously pushing the model toward an unrealistic 8.6-year end of life. Checked against a real, full-year solver run and projected out to 20 years, this A123 ANR26650M1 pack is dominated by calendar ageing (wear from simply sitting, not from use) and does not get close to its 80% health threshold within 20 years. This result fairly reflects the model as built, but it should be read together with the reasons in Section 6 and the improvements in Section 7, especially the cell-size assumption and the constant-temperature assumption, before being used for any warranty or lifetime-cost decision.
